

April 2026

TIMEBOX TESTING LAB: A FAULT INJECTION FRAMEWORK

A Full-Stack Testing System Using FastAPI, Pytest, and
Playwright

HAMIZ SIDDIQUI
MS Computer Science

Problem and Project Idea

Why this project

Flaky behavior usually comes from timing, ordering, stale state, or small hidden assumptions that only show up once the app is pushed out of its clean baseline.

Project direction

Instead of copying a huge research-style proposal, I built a smaller fault-injection lab where bugs can be turned on and observed immediately.

What makes it useful

The same dashboard supports normal task actions, bug activation, scenario presets, test automation, and a visible browser demo.

Final app focus

Simple task workflow
Controlled fault profiles
Session-safe behavior
Visible demo artifacts
Readable test outcomes

Tech stack at a glance

BACKEND

FastAPI

web framework + routing

SQLite

zero-config persistence

itsdangerous

signed session cookies

uvicorn

ASGI server

~450 lines in main.py

FRONTEND

Jinja2 templates

server-rendered HTML

Vanilla CSS

~600 lines, custom design

Vanilla JavaScript

~40 lines, no framework

4 templates

base, login, register, dashboard

No build step, no bundler

TESTS & TOOLING

Pytest

test runner + fixtures

Playwright

real-browser automation

FastAPI TestClient

in-process API calls

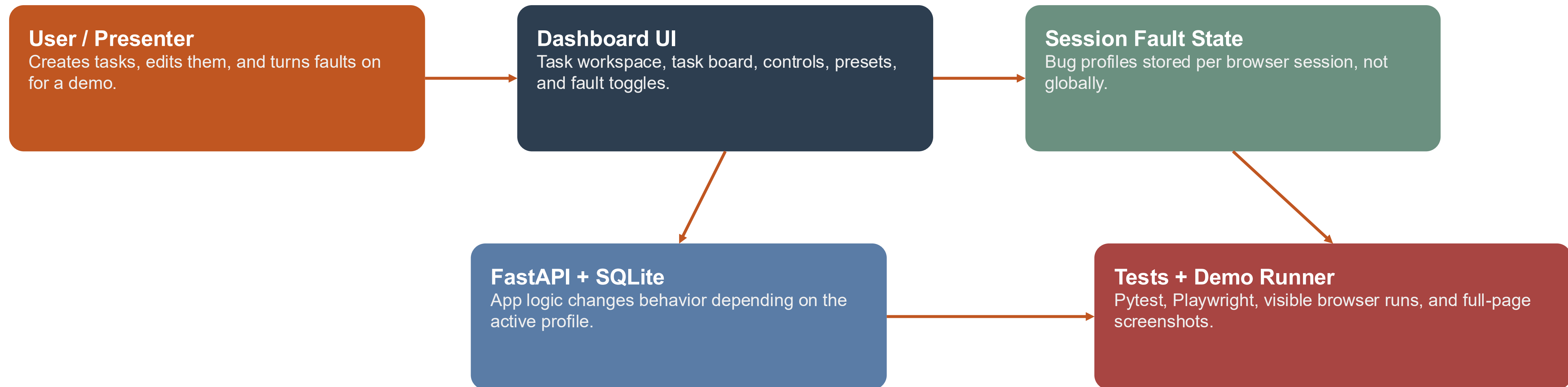
demo_browser_run.py

visible scenario walkthrough

13 automated tests in total

Deployment: render.yaml + Procfile — ready to deploy to Render's free tier.

Architecture and flow



Features and workflow

1. Task workspace

Create new tasks and edit existing ones from the same form.

Keeps the app realistic enough for repeatable demos.

2. Session fault control panel

Turn fault profiles on and off directly from the dashboard.

One browser session can be buggy while another stays clean.

3. Demo presets

Search, timing, and sorting presets activate grouped scenarios with one click.

Makes the presentation flow much faster.

4. Task board

Shows completion state, overdue status, due time, and quick actions.

Also gave us a clean place to validate edits and state changes.

5. Scroll restore

After clicks, the page returns to the same spot instead of jumping to the top.

Small fix, big improvement during demos.

6. Full-page screenshot capture

Demo images now show the whole page after the app settles.

That made the report and slides much more usable.

Faults and demo presets

Fault profiles

Slow UI
Duplicate create bug
Unstable sort bug
Search regression
Dashboard cache bug
Midnight boundary

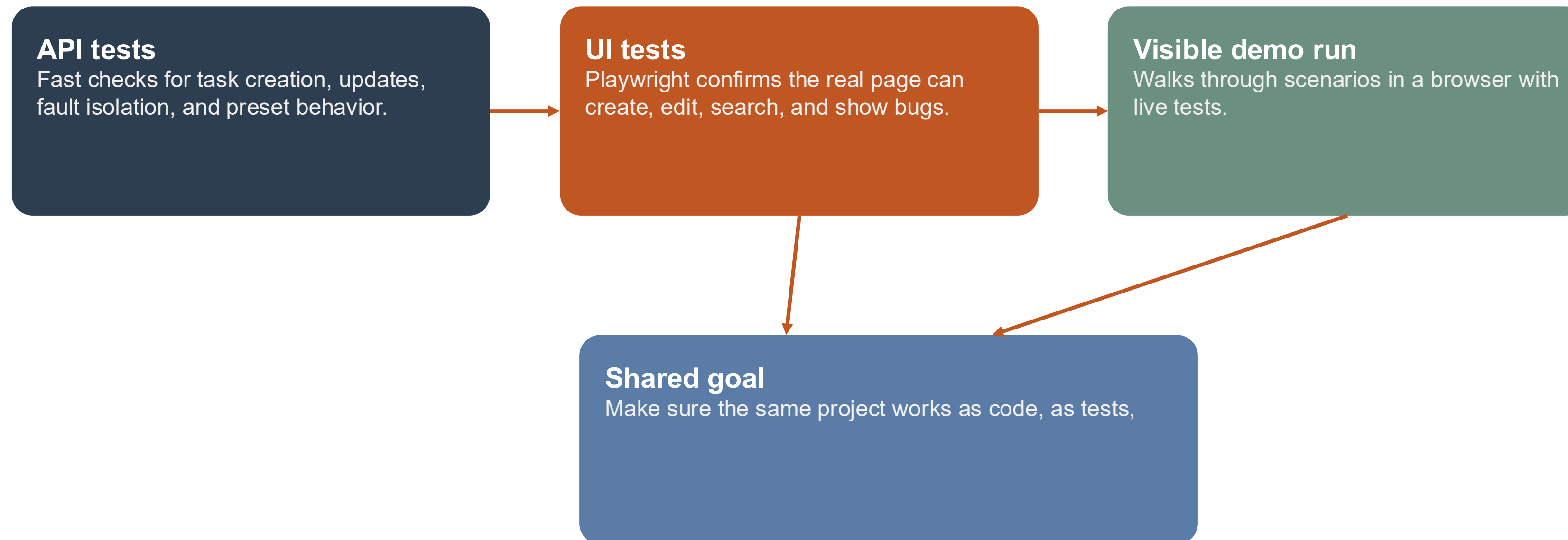
Preset examples

Search Demo → search regression
Timing Demo → slow UI + midnight boundary
Sorting Demo → unstable sort

Why presets mattered

They let me show a full scenario quickly instead of toggling multiple bugs manually every time.

Testing strategy



Automated checks that passed in the final version:

- **9 API tests**
session faults, updates, presets, duplicate create, search behavior, and boundary logic.
- **4 UI tests**
happy path, edit flow, duplicate bug visibility, and preset-based search failure.

Reproducibility design

01

Frozen clock

Set in the test env:

`TIMEBOX_FIXED_NOW = 2026-04-17T12:00:00`

Pins the app's view of "now" so due-date logic and the midnight-boundary fault produce identical results every run.

02

Reset hooks

Called in every pytest fixture:

`reset_db()`
`reset_fault_profile()`

Each test starts from a clean database and a clean fault profile, so order-of-execution never changes the outcome.

03

Session isolation

Verified by a dedicated test:

`test_fault_profiles_are_isolated_per_session`

Two TestClient instances, two signed session cookies. One client's active faults never leak into the other's view.

Test coverage breakdown

API tests

9 tests • tests/test_api.py

Auth

Unauthorized request to /api/tasks returns 401.

Validation

Empty task title rejected with 400.

CRUD

Create-then-read returns the task. Update via PUT changes title, description, and completion state.

Fault behavior

Duplicate create produces 2 rows. Search regression drops lowercase matches. Midnight boundary shifts status.

Session isolation

Two TestClient instances — client A's active faults stay invisible to client B.

Preset semantics

Applying a preset replaces the existing profile rather than merging with it.

UI tests

4 tests • tests/test_ui_playwright.py

Happy path end-to-end

Register → log in → reset faults → create task → search → see result.

Edit flow

Create a task, open it in edit mode, change title + description + due date, and confirm the old row is gone.

Duplicate fault visibility

Toggle duplicate_create_bug, submit one form, assert 2+ rows actually appear in the DOM.

Preset-based search failure

Apply search_demo preset, create "Alpha Task", search for "alpha", assert zero rows (case-sensitive regression).

Experimental results

9

API tests passed

4

UI tests passed

Representative outcomes

Duplicate create → one submit creates two rows.

Search preset → 'alpha' does not find 'Alpha Task'.

Timing preset → UI slows down and due-date edges become easier to challenge.

Sorting preset → ordering becomes less stable and more surprising.

What improved the most

Session-based faults made behavior easier to isolate.

UI controls made the lab much easier to present.

Task editing made the app feel less fake.

Issues we ran into and how we fixed them

- **API-only controls felt hidden**

Fix: moved fault toggles into the dashboard and kept the API mainly for automation.

- **Global fault state was messy**

Fix: stored the fault profile in the browser session so demos do not interfere with one another.

- **The app needed a better workflow**

Fix: added edit-task support instead of forcing everything through create/delete.

- **Button clicks jumped back to the top**

Fix: added a lightweight scroll-restore script so the page stays near the action.

- **Demo screenshots looked incomplete**

Fix: capture full-page images after the page settles and scrolls back to the top.

Why those fixes matter

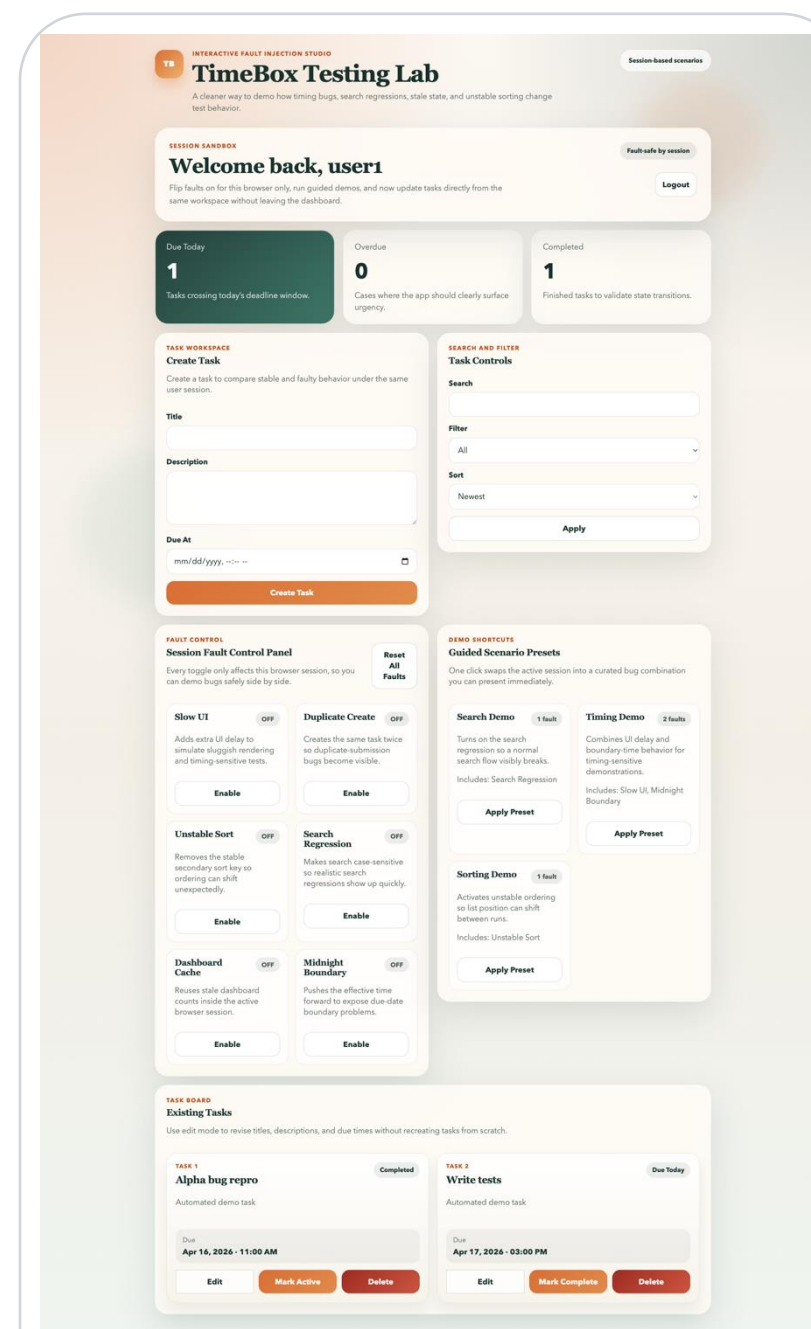
Without them, the project technically works but feels unfinished.

With them, the app, tests, screenshots, and presentation all tell the same story.

That ended up being one of the biggest lessons of the whole project.

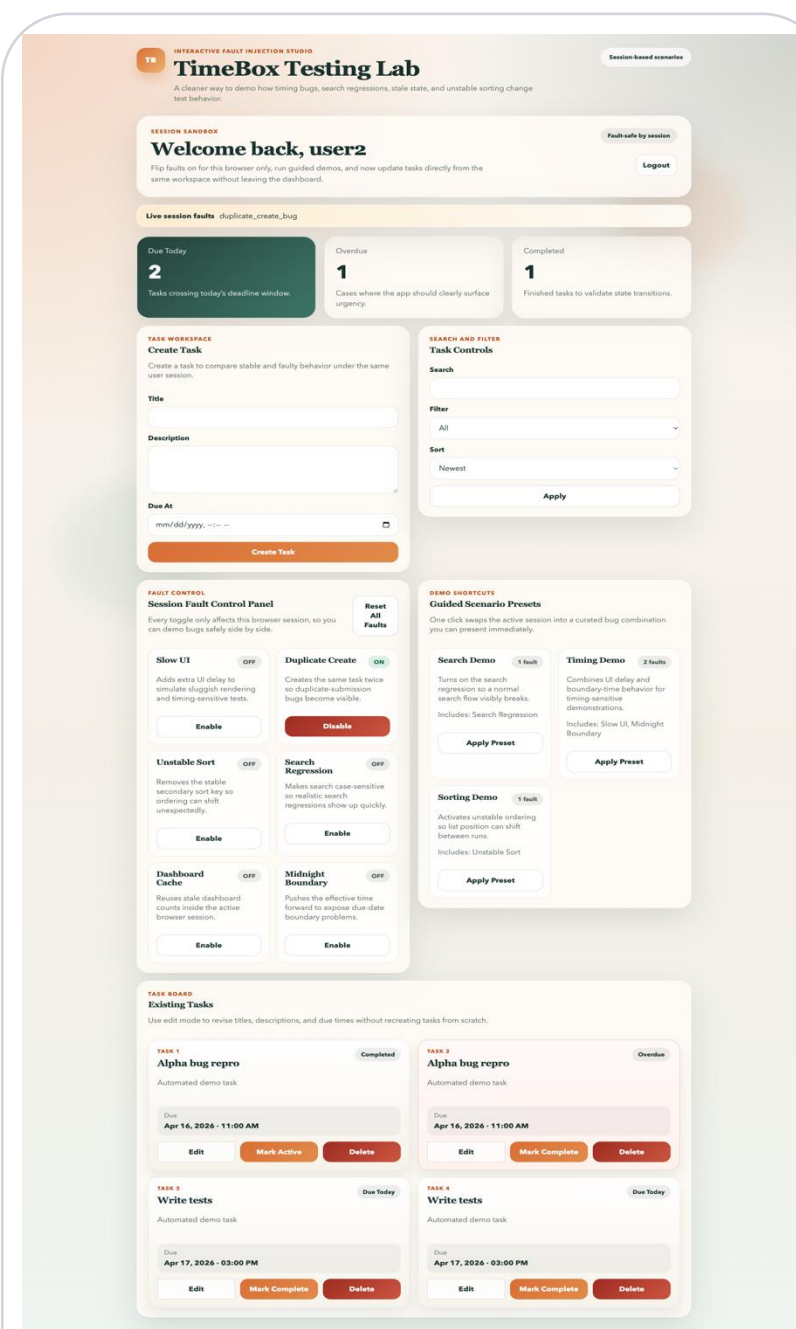
All 5 demo scenarios

demo_browser_run.py launches a visible browser and captures a full-page screenshot of each scenario.

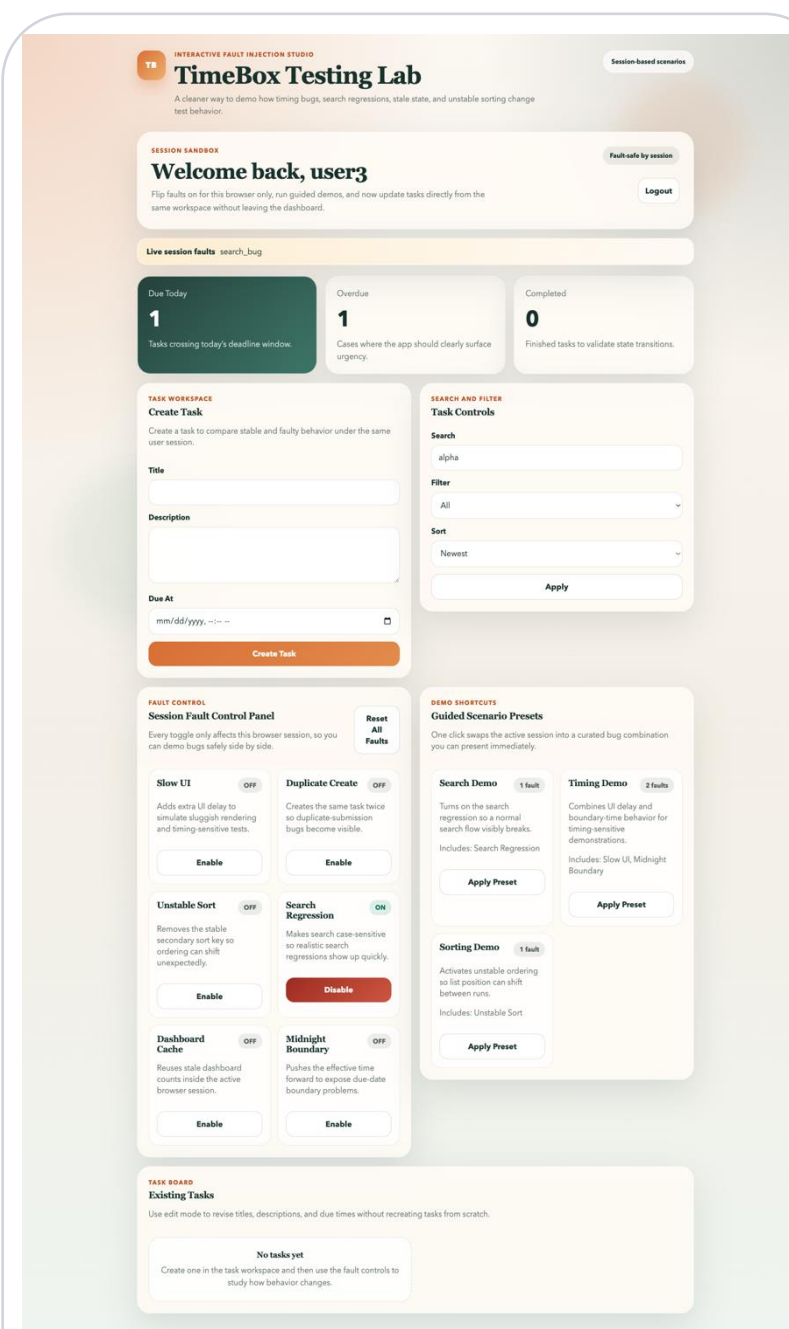


1. Happy path

no faults active

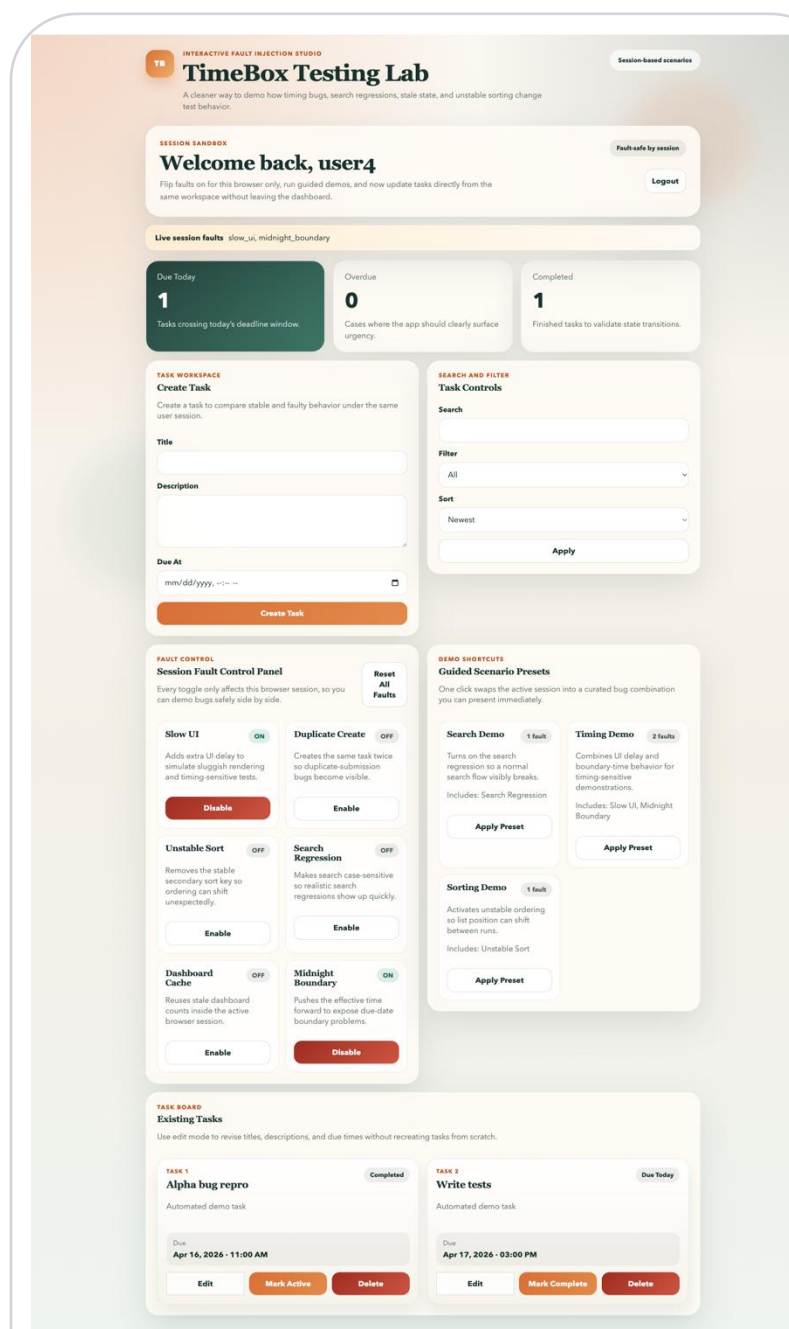


2. Duplicate create



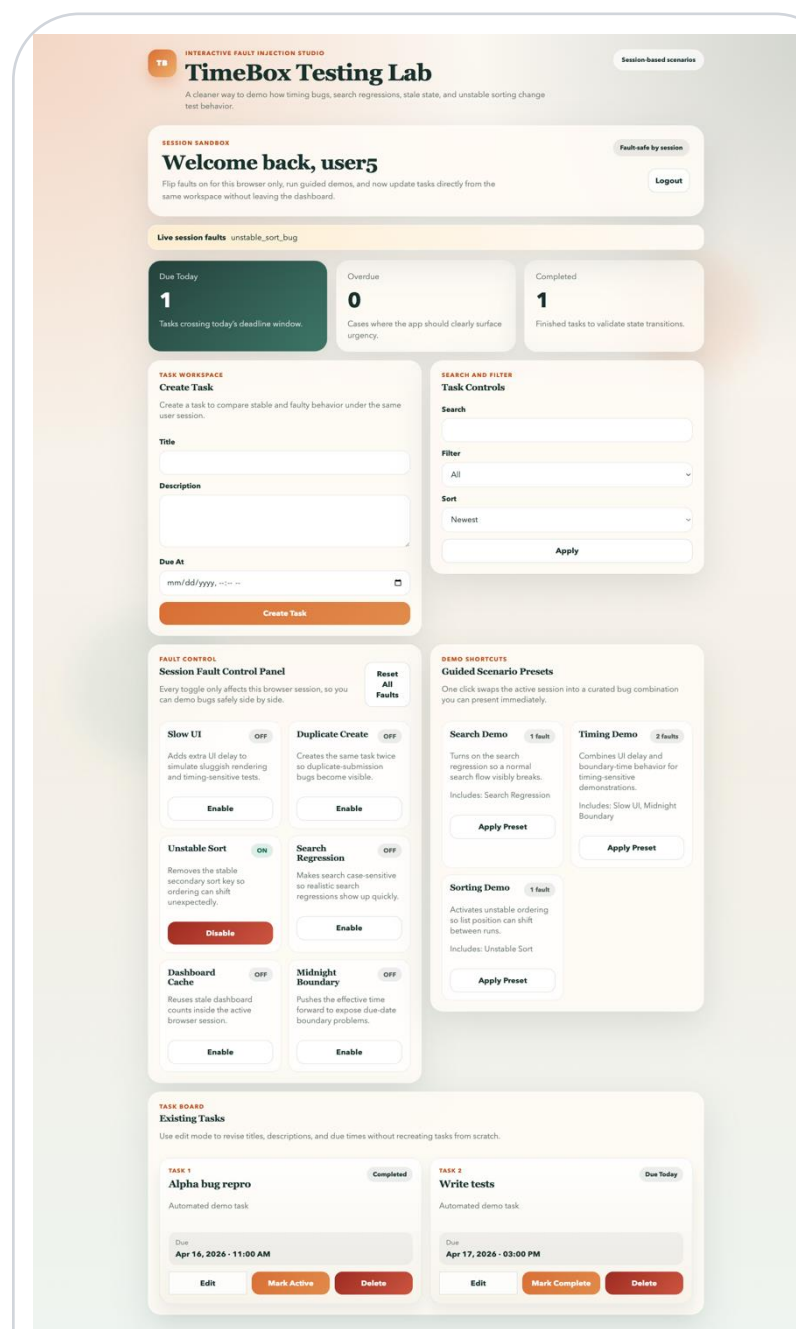
3. Search demo

search_bug via preset



4. Timing demo

slow_ui + midnight_boundary



5. Sorting demo

Takeaways and demo commands

Main takeaways

A small app can still show meaningful flaky behavior if the fault profiles are chosen carefully.

Session-based controls were one of the most important design improvements.

Polish matters: editing, scroll behavior, and better screenshots made the project much easier to present.

The final version connects features, tests, and demo artifacts in one place.

Future Work

CI/CD integration (GitHub Actions)

Multi-user testing scenarios

More fault types (network latency, API failure)

Metrics dashboard (failure rates, timing)

THANK YOU.

- FastAPI + SQLite — session-scoped fault injection
- Pytest + Playwright — API, UI, visible browser demo
- Six fault profiles, three demo presets — repeatable bug scenarios

Hamiz Siddiqui — MS Computer Science

TimeBox Testing Lab • April 2026